Checkpoint Feedback Assessment

Async Checkpoint Feedback — Assessment & Recommendation

Prepared: 2026-06-01 · **Reviewer:** AI assist + review squad (red-team, critic, fact-checker, outsider, synthesizer) **Input:** `checkpoint_feedback_spec.md` / `.pdf` / `.pptx` (Hume, v1) **Method:** every load-bearing claim ground-truthed against the live codebase (`scripts/`, `supabase/`), then stress-tested by four independent reviewers and synthesized.

Verdict: RESCOPE (do not ship as written; do not reject)

The core idea is **sound**: seal a judgment before the outcome is known, grade it asynchronously when it ripens, and let lessons back in only when enough same-version data has earned them. The two-clock split (learn slow / sell fast) and the firewall framing are correct instincts.

But the spec ships as if the infrastructure doesn't exist (≈70% of it already does) and as if the sample-size math will work this year (it won't). It also carries three concrete technical blockers that would silently corrupt data if migrated as-is. The right move is to **rescope to a ~2-page extension of the existing prediction stack**, build the 2–3 genuinely novel pieces, and drop or defer the rest.

What already exists (the spec's central blind spot)

The spec proposes `thesis_predictions` + `run_checkpoint.py` + a new `thesis_outcomes` as if from scratch. In fact:

Spec concept	Already in the codebase
Sealed prediction record at run time	`prediction_log` table + `prediction_logger.log_prediction()` (UNIQUE `ticker,run_id`; stores target band, archetype, valuation method, `context_inputs` jsonb)
Recording actuals at a horizon	`prediction_outcomes` + `prediction_logger.record_price_outcome(prediction_id, days_elapsed, actual_price)`
Forward-price grading at T+30/90/180	`theses` + **existing** `thesis_outcomes` (1:1 via `thesis_id`) + `thesis_calibration` view
Analysis to hook onto	`socratic_analyses` (model_a/b/c + disagreements; `run_socratic.py` inserts ~line 1050 and returns the id)
	`feedback_loop._fetch_historical_price()` (yfinance window) — **not** `refresh_prices.py`, which is current-price only

Spec concept	Already in the codebase
Historical close at an arbitrary past date	
N-gated weight adjustment	`feedback_loop.py` (`MIN_SIGNALS_FOR_ADJUSTMENT = 10`, Bayesian beta-binomial shrinkage, `PRIOR_STRENGTH = 50`)

The spec never mentions `prediction_log` / `prediction_outcomes`. That omission is the root of most of the problems below.

What is genuinely new and worth building

Only three elements have no equivalent today, and all three are valuable:

1. **`system_version` stamping** — tag every prediction with the git hash / version that produced it. This is the load-bearing new idea: it lets old lessons be archived rather than misapplied. Add it as a column on `prediction_log`, not a new table.
2. **`reasoning_fingerprint`** — `{dissenting_model, unresolved_questions, conviction}`. Enables *process* insight ("Model B ran hot on the last 4 regime-shift names"), not just *outcome* insight. Cheap to capture now; partially derivable from `socratic_analyses.disagreements`.
3. **The sample-size + version insight gate** — applying the project's own Hard-Gate / Contextual / Informational taxonomy to the feedback layer itself. The discipline is right; the thresholds need fixing (below).

Everything else is either already built, restated existing architecture in new vocabulary, or premature.

Blockers (P0 — would corrupt data if built as written)

1. **`thesis_outcomes` name collision.** A table by that exact name already exists with an incompatible schema (keyed `thesis_id INT` → `theses`). `CREATE TABLE IF NOT EXISTS` would silently no-op and leave the old schema; the grader then writes columns that don't exist (hard-fail or silent strip), or a drop-and-recreate destroys the `thesis_calibration` view. *Fix: introspect the live table first; use a distinct name (`thesis_checkpoint_outcomes`) or extend `prediction_outcomes`.*
2. **Foreign-key type mismatch.** Spec declares `analysis_id uuid fk -> socratic_analyses`, but that table's PK is `bigint GENERATED ALWAYS AS IDENTITY`. A uuid FK fails on migration. *Fix: `analysis_id bigint`.*
3. **Immutable grades against an unlogged date.** Grading needs the close at `sealed_at + H`. `_fetch_historical_price()` returns the first price in a forward window with no record of *which* calendar date — near holidays/halts it can silently grade against the wrong day, and the grade is immutable. *Fix: explicit date-pinning + store the actual date used (the old `thesis_outcomes` already has `price_t30_date` for exactly this reason).*

Fact corrections (claims that don't match the code)

- **"MIN_SIGNALS_FOR_ADJUSTMENT = 10 is a blocking gate"** — *misleading.* It's a soft logging-skip; weights are Bayesian-blended (at n=10 the data contributes ~17%), not hard-gated.
- **"gates (5, 20) map to existing `MIN_SIGNALS=10` / N≥20 thresholds"** — *contradicted.* The existing constant is 10, not 20; there is **no** N≥20 weight threshold in `feedback_loop.py` (the only N≥20 is a frontend comment in `socratic.sql`). The spec's "no new magic numbers" self-claim doesn't hold.
- **"the loop is synchronous and blocking — nothing updates until outcomes settle"** — *misleading.* The feedback step runs in-pipeline but is exception-guarded and does **not** gate model generation; outcome settlement is already asynchronous. The latency complaint is real; the "blocking" framing is not.
- **"no sealed prediction exists today"** — *contradicted* (see `prediction_log`). The spec's new *fields* are novel; the record is not.

Dependencies the spec cites — **D5** (implied_expectations, unbuilt), **F9** (re-arm at ≥20 outcomes), **E2** (signal-store census) — all check out against the roadmap.

The one conceptual fix that matters most

Version-scoping as written makes the system permanently inert. §5 hard-resets a component's track record to N=0 on any major judgment-layer change. With 6 watchlist names, a ~30-day horizon, and frequent Socratic/prompt rewrites (≥7 between the AXON seal and today), the system would **never** accumulate 20 clean same-version outcomes — the "Corrective" tier never activates and the whole apparatus reduces to a logger.

Recommended resolution: treat `system_version` as a **covariate, not a partition key**. Record it on every prediction; *pool* observations for sample-size counting. Add a `version_cohort_break` flag marking the first prediction after a major change so the calibration view *can* split by cohort when curious — but count N across all records, and run a sensitivity check ("does accuracy differ materially across cohorts?") rather than discarding history. This is how clinical trials handle protocol amendments. Set the Corrective threshold at **N≥10** (matching the real existing constant), not 20.

The honest June 8 plan (AXON)

The "hard deadline" is real for a tiny ritual and theater for the rest. AXON was entered ~2026-05-09, *before* this system exists. You cannot cryptographically "seal" it now without either capturing today's price (look-ahead) or hand-reconstructing the 05-09 price (unverifiable, and `sealed_hash` can't detect pre-seal contamination). So:

1. **Today (~30 min):** write `data/axon_seal_2026-05-09.md` — the 05-09 ref price, the thesis, and an explicit note that this is an honest reconstruction predating the formal system, not a tamper-proof seal.

2. **June 8 (~30 min):** fetch the close on 2026-06-08, compare to ref price via `feedback_loop._fetch_historical_price()`, and write a one-paragraph post-mortem: was the directional call right, and why.
3. **Do not** ship any new table/schema before June 8 — the collision risk outweighs the value of a formal record at N=1.

Drop `sealed_hash` entirely (security theater on a single-owner DB with no independent verifier).

Prioritized action list

P0 — blockers (do before any schema migration)

- Resolve the `thesis_outcomes` name collision (introspect → rename to `thesis_checkpoint_outcomes` or extend `prediction_outcomes`). ~1 h
- Fix `analysis_id` FK type to `bigint`. ~0.5 h
- Add explicit date-pinning + `actual_date_used` logging to historical-price grading. ~2 h

P1 — high value, low risk

- `ALTER TABLE prediction_log ADD COLUMN IF NOT EXISTS system_version text, reasoning_fingerprint jsonb`; populate `system_version` from `git rev-parse --short HEAD`. ~2 h
- Replace the N=0 hard reset with a `version_cohort_break` covariate flag; set Corrective threshold to N≥10. ~half day (design + view)
- Write the honest AXON reconstruction note today; run the June 8 post-mortem. ~1 h total

P2 — defer

- `implied_expectations` capture — blocked on D5; scope separately.
- Live `[CALIBRATION]` insight injection — defer until ~N≥10 pooled (realistically Q4 2026); until then keep it informational-only and surfaced to you, never altering numbers.
- A standalone `run_checkpoint.py` — only if the extended `feedback_loop` grading path proves insufficient; the missing piece is date-pinning, not a new process.

Recommended answers to the open questions

- **H1 (cron cadence):** Agree with the spec — daily grading; keep fast-sell inside D5 on its own clock. Preserves the two-clock split.
- **H2 (`[CALIBRATION]` placement):** Keep it **out** of Model B's own context until N is meaningful. Letting a model see its own fingerprint at N≤5 invites gaming with zero statistical upside. Surface it to *you*, not to the models, this year.
- **H3 (per-archetype vs pooled):** Pooled — and given the version-covariate fix, pooled is now doubly correct. Per-archetype only once each archetype clears N≥10.
- **H4 / G-new (clock-resetting threshold):** This is the most important decision, and the version-as-covariate fix largely dissolves it — nothing "resets," so you can evolve the judgment layer freely while still flagging cohort breaks. Define a "major change"

narrowly (analyst scoring math, Socratic model roles, corpus-callosum logic) purely as the flag's trigger, not as a data-discarding event.

Bottom line

Keep the idea, keep `system_version` and `reasoning_fingerprint`, keep the gate *discipline*. Cut the new tables, the hash, and the hard version reset. Build the three novel fields onto the stack you already have, fix the version-vs-sample-size tension so the loop can actually mature, and treat June 8 as an honest one-paragraph post-mortem rather than a schema deadline. Rescoped this way it's roughly a day of work that strengthens what exists, instead of a parallel system that does nothing new until autumn.

Reviewers converged independently on the same core findings: sound concept, duplicated infrastructure, unreachable calibration tier, and the `thesis_outcomes` collision as the hardest technical blocker. Full ground-truth evidence in `scripts/feedback_loop.py`, `scripts/prediction_logger.py`, `supabase/2026-04-26_prediction_log.sql`, `supabase/2026-05-05_thesis_outcomes.sql`, `supabase/2026-05-15_socratic.sql`.